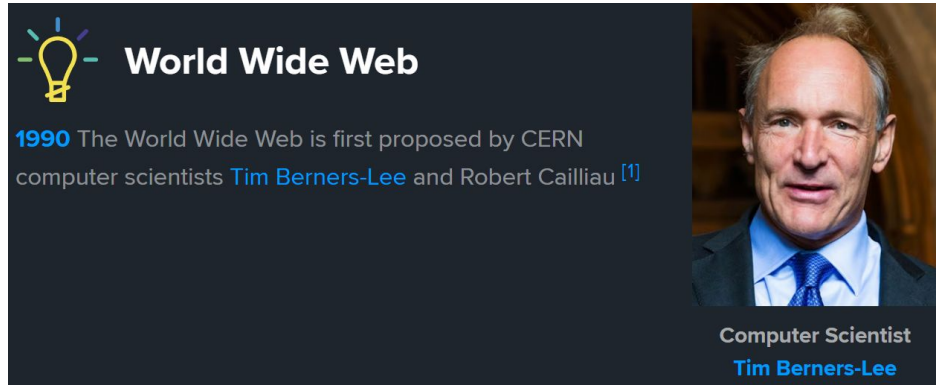


COMS W4156 Advanced Software Engineering (ASE)

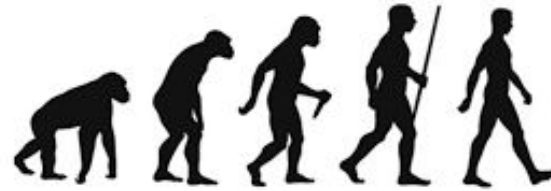
November 12, 2024

On this day ...



Agenda

1. Refactoring
2. Refactoring demo: Vrinda Ahuja
3. 6156 advertisement (Spring 2025)
4. Reminders



Refactoring

Improving the Design of Existing Code

What is Refactoring?

I've mentioned "refactoring" a few times in earlier lectures, but not always compliant with its technical meaning

Technically, refactoring is a systematic approach to improving code quality through *semantics-preserving* transformations, not an arbitrary codebase reorganization

Semantics-preserving means does not change functionality

[Refactoring operations](#) (or recipes) should be short sequences of code edits that do not change the functionality of the program as a whole, so *unchanged system-level functional tests should get exactly the same results* before and after each refactoring operation



The individual edits in the sequence may indeed change system functionality, or break it entirely, so code edit sequences must be atomic = all or nothing

Refactoring Example

Let's say we have

```
veryLongMethod (...) {  
    many lines of code  
}
```

The body of the method might be hard to read and hard to modify, e.g., to fix bugs or add features

So we might want to refactor to

```
veryLongMethod (...) {  
    shorterMethod(...);  
    if ( someCondition(...) ) {  
        anotherShorterMethod(..., aMethod(...),  
...);  
    }  
}
```

If the new methods have well-chosen mnemonic names, this version is probably easier to read and modify, even if the new methods are not used anywhere else (but, ideally, extracted methods are reusable elsewhere)

Why Refactor?

To make the code easier to read

To make it easier to fix a bug

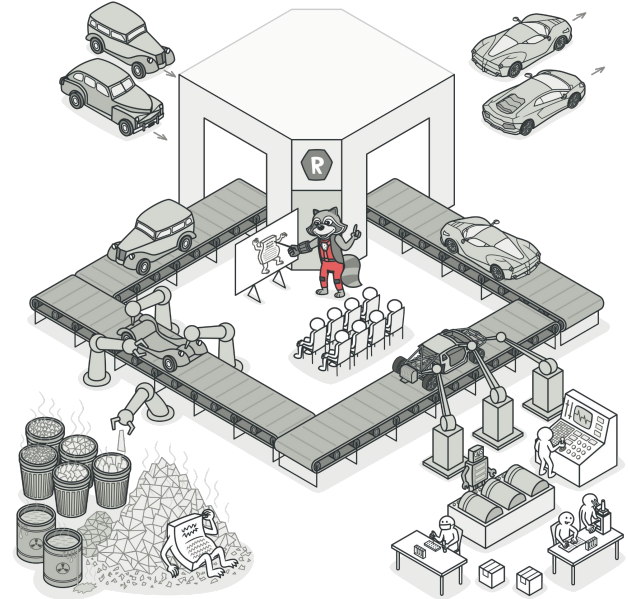
To make it easier to add a feature

To remove code smells

To be compliant with SOLID principles

To introduce a design pattern

...



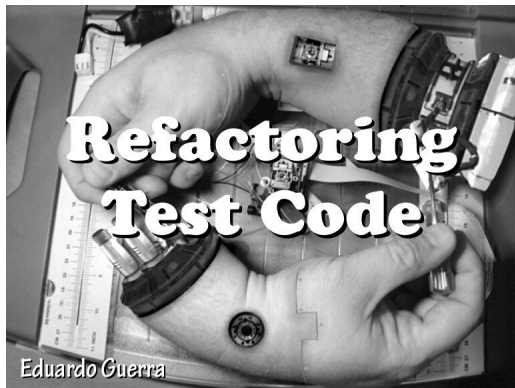
Testing During Refactoring

Ideally, all system-level functional tests are indeed rerun before and after each refactoring operation (the whole edit sequence, not each individual edit) to verify that nothing was broken - if any error was made during the editing sequence, then it's much easier to back out of one refactoring than many

But it may not work to rerun unit and integration tests after each refactoring because the tests themselves may need to be refactored to match the code changes - refactoring can add units, remove units and move functionality between units



Refactoring Example Continued



Which functional test cases should be changed, added or removed?

Let's say we have

```
veryLongMethod (...) {  
    many lines of code  
}
```

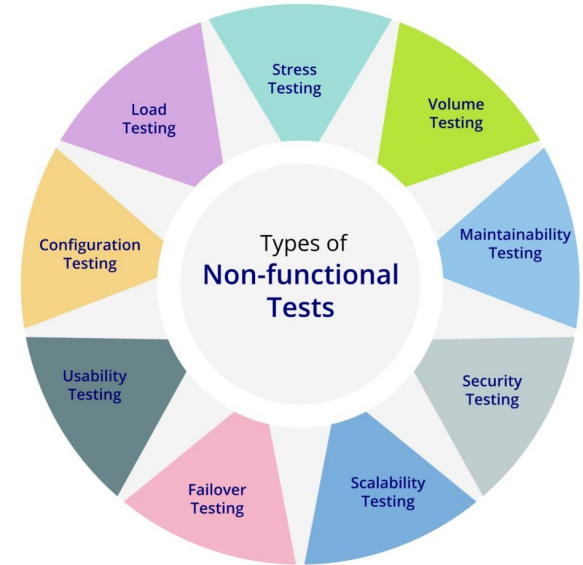
and refactor to

```
veryLongMethod (...) {  
    shorterMethod(...);  
    if ( someCondition(...) ) {  
        anotherShorterMethod(..., aMethod(...), ...);  
    }  
}
```

What Do I Mean By Functional Tests?

Most likely, all the testing you have done so far for the miniproject and your team project have been functional tests, testing the functionality (or semantics) of the code

But if you were implementing a real-world service, you might want to test that it works on different versions of the operating system or even on different operating systems (compatibility testing). You might want to test that it can handle 10, 100, 1k, 10k, 100k clients at the same time (scalability testing). You might want to test how long it takes (on average) to respond to 10, 100, 1k, 10k, 100k clients with simple, typical or extreme requests (performance, load or stress testing)



Simple Refactoring Example: Renaming

Why rename an identifier? The class/method/variable name is not descriptive enough, something new has been introduced requiring existing code to use a different and/or more specific name,

...

Conventional find-and-replace editing operations are text-based and could unintentionally change unrelated items with same name (or name subtoken), but most IDEs support smarter identifier renaming

```
Server = new Server(path, port, endpoint);
```

```
Server.init();
```

```
Server.run();
```

```
Server = new WebSocketServer(path, port,  
broadcastingServerEndpoint);
```

```
Server.validate();
```

```
Server.run();
```



Renaming Refactoring

When renaming a field, want to rename all uses and its getter/setter methods

When renaming a method, want to rename all calls and all overridden/implemented methods in subclasses throughout the codebase

When renaming a class, want to rename all uses of the class throughout the codebase -- and possibly constructors/destructors and other parts of the code to match



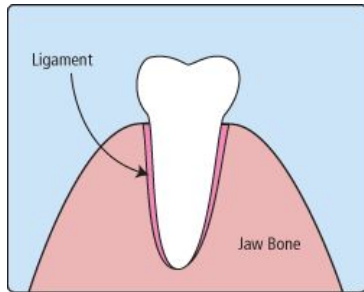
Extract Method Refactoring

Problem - You have a code fragment that appears multiple times (and is longer than some minimum threshold. e.g., 3 lines, 5 lines, 10 lines...)

Solution - Move this code to a separate new method and replace the old code with a call to this method

```
void printOwing() {  
    printBanner();  
  
    // Print details.  
  
    System.out.println("name: " + name);  
    System.out.println("amount: " +  
getOutstanding());  
}
```

```
void printOwing() {  
    printBanner();  
  
    printDetails(getOutstanding());  
}
```



```
void printDetails(double outstanding) {  
    System.out.println("name: " + name);  
  
    System.out.println("amount: " +  
outstanding);  
}
```

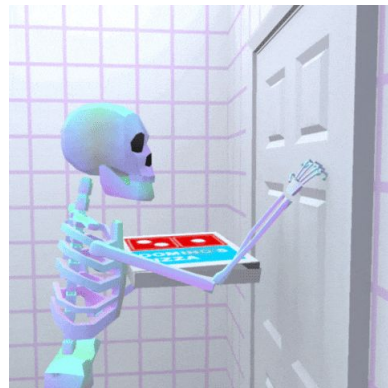
Inline Method Refactoring

Problem - When a method body is needed but the method is not, “why do we have this method?” (Only if not redefined in subclasses, then need to keep it)

Solution - Replace calls to the method with the method’s content and delete the method itself

```
class PizzaDelivery {  
    // ...  
    int getRating() {  
        return moreThanFiveLateDeliveries() ? 2 : 1;  
    }  
    boolean moreThanFiveLateDeliveries() {  
        return numberOfLateDeliveries > 5;  
    }  
}
```

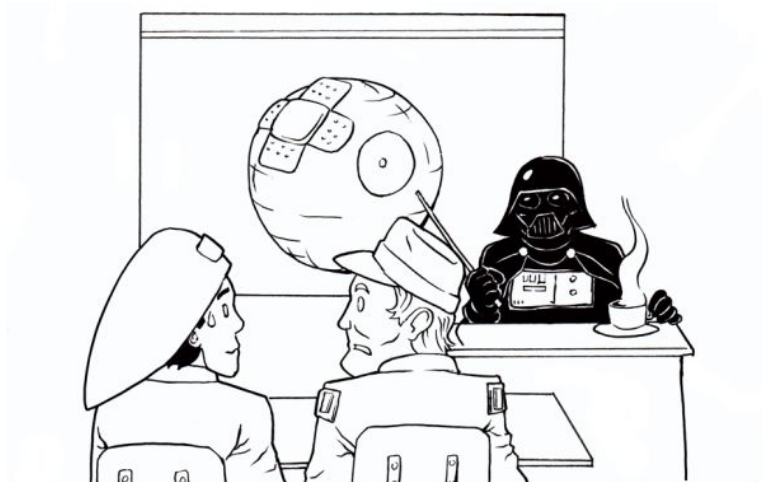
```
class PizzaDelivery {  
    // ...  
    int getRating() {  
        return  
        numberOfLateDeliveries > 5  
        ? 2 : 1;  
    }  
}
```



Refactoring vs. Other Code Changes

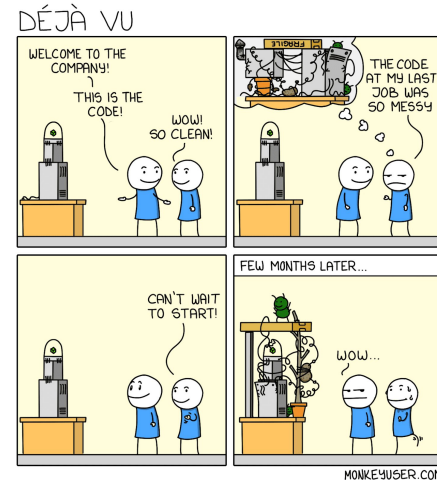
Refactoring should not be included in the same commit as bug-fixes.

Refactoring should be done in a separate commit before a bug-fix.



Refactoring should not be included in the same commit as new features.

Refactoring should be done in a separate commit before a new feature.



Some Categories of Refactoring Operations

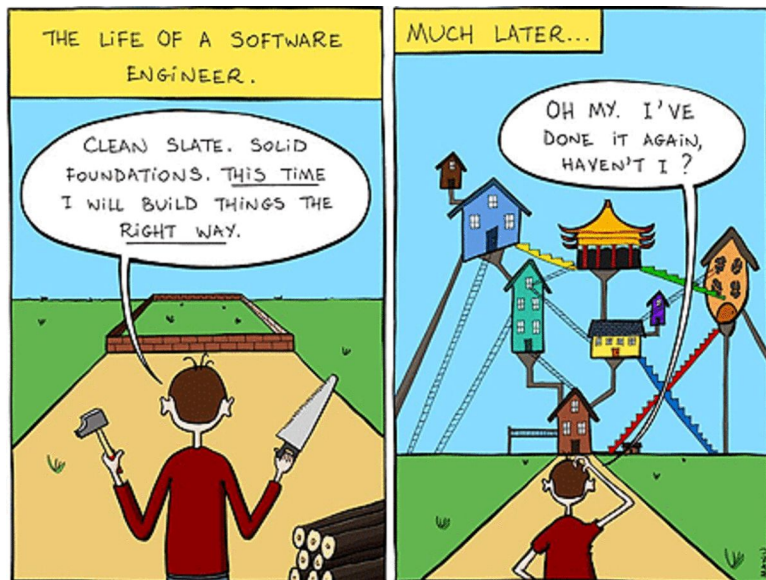
Composing Methods - Excessively long methods are the root of all evil. Methods that are hard to understand are hard to change. Streamline methods, remove code duplication, and pave the way for future improvements

Moving Features between Classes - Even if you have distributed functionality among different classes in a less-than-perfect way, there's still hope! Safely move functionality between classes, create new classes, and hide implementation details from public access



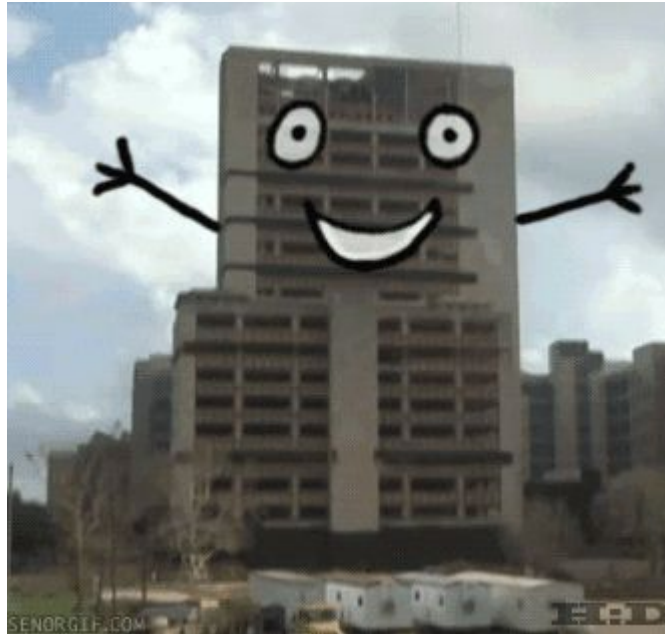
Agenda

1. Refactoring
2. Refactoring demo: Vrinda Ahuja
3. 6156 advertisement
4. Reminders



Vrinda Ahuja Refactoring Demo

[demo slides](#)



Agenda

1. Refactoring
2. Refactoring demo: Vrinda Ahuja
3. 6156 advertisement
4. Reminders



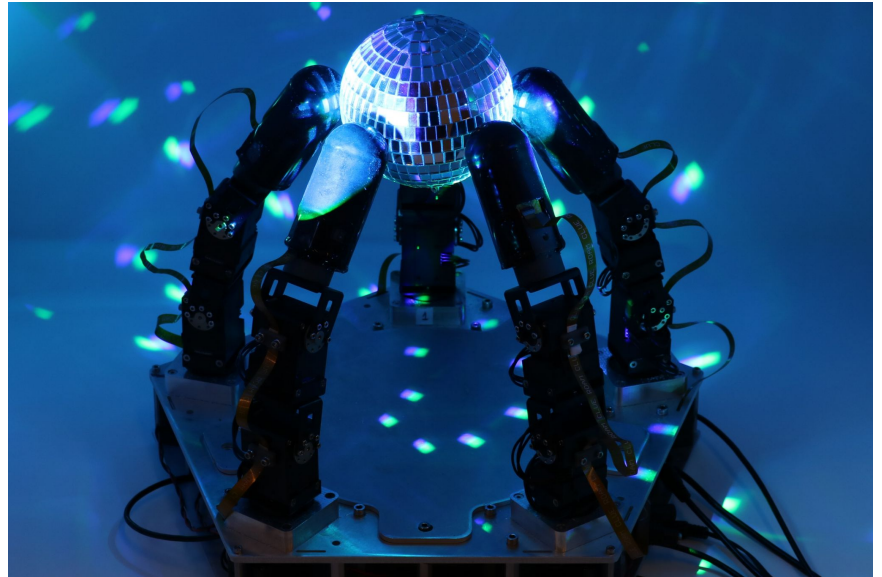
COMS E6156 Topics in Software Engineering

6156 is not "more" 4156 and is not "more advanced" 4156

6156 is a “topics” course, like 6998, where each section has a different topic

COMS E6156 Topics in Software Engineering. 3 points.

Topics in Software Engineering arranged as the need and availability arises. Topics are usually offered on a one-time basis. Since the content of this course changes, it may be repeated for credit with advisor approval. Consult the department for section assignment.



But 6156 Has Only One Section Now

Once upon a time, Prof. Donald Ferguson taught his very popular Cloud Computing course as a section of 6156, but now his course has its own name and number

([COMS W4153 Cloud Computing](#))



⇐ not me



⇐ not covered

What is My Section of 6156 About?



6156 is still a "topics" course, but I do not choose the topic, the students do

I may provide a suggested or default topic, e.g.,

- LLM-based translation from C to idiomatic safe Rust
- Protecting containerized applications from compromised libraries
- Generating unit tests from buggy execution traces to test candidate fixes

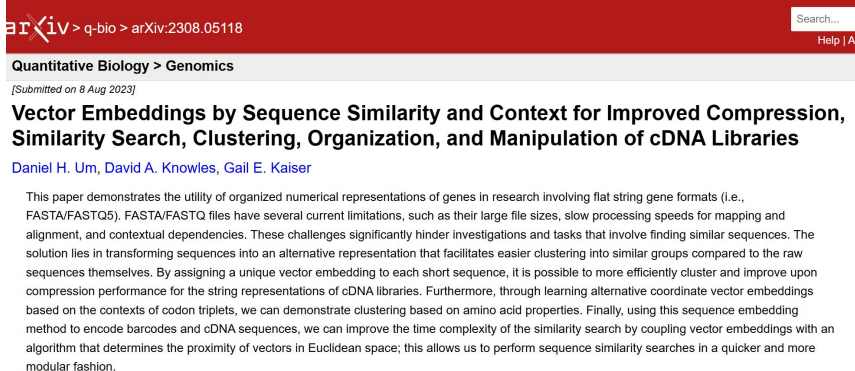
But each student can choose their own topic, with my approval, different from all the other students' topics

Every topic should somehow be relevant to software engineering - broadly construed

Examples of "Broadly Construed"

computational biology genomics:

Accepted to appear in the Journal of Computational Biology and Chemistry, Elsevier, 2024



programming language compiler optimization:

Lua Workshop 2016: A Sun For the Moon - A Zero-Overhead Lua Abstraction using C++ - ThePhD



6156 Workload

Students

- read, present and discuss research papers
- write a midterm literature review paper; and
- conduct and report a final research project



No lectures after first week or so, thereafter mostly student presentations and demos

Students present research papers from curated list (mostly recent best paper awards) or suggest their own (with approval)

Individually written literature review paper due shortly before spring break

Individual or team (students' choice) research project report and demo due at end of semester

Who Should Take 6156

Oriented towards students who want to become (or already are) researchers or technology leaders

Researchers read research papers by other researchers to *keep up to date* on their research field and to *get ideas* for their own research. They do research to try to *solve interesting problems* and prototype solutions that may, eventually, lead to real-world impact



Technology Leaders (not necessarily the same as "tech leads") usually work in industry or government and read research papers to find ideas that could *potentially* be productized, improve products or productivity, or inform company culture, industry standards and/or government policies. They may also do small research projects to determine how *practical* it is to realize that potential for real-world impact *now*

Some Recent Student-Chosen Topics

Fault Localization Techniques for Deep Neural Networks

GAN-based Inpainting Algorithms

MLIR: Design and Impact

MLOps Challenges and Solutions

Mixed Reality Socializing

Sparse Matrix Support in LLVM

Bias in AI Image Generation

Transformers for Masked Facial Recognition

Gaze-Controlled VR Game



Questions?



Agenda

1. Refactoring
2. Refactoring demo: Vrinda Ahuja
3. 6156 advertisement
4. Reminders



Coming Up

[Second Iteration](#) due November 25
(Monday before Thanksgiving)

[Second Iteration Demo](#) due December 6
(Friday the week after Thanksgiving)

Second Assessment Friday December 6
12:01AM to Monday December 9
11:59PM (last day of classes and the day
before university "study days")

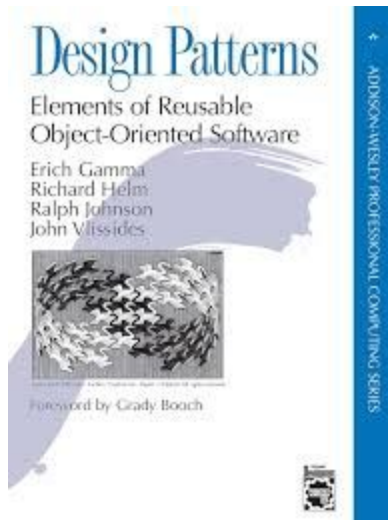
[Demo Day](#) Monday December 16 10AM to
4PM (Monday of finals week)



Thursday

Docker demo

(Some) Design Patterns



Ask Me Anything

